Lesson 5 — Cloud Infrastructure and Virtualization

INTE 22253 | Source: Lecture 05 "Hypervisors, Virtual Machines, Containers, and Cloud Data Centre Architecture" (41 slides) | Builds on Lessons 2–3; Lesson 8 goes deeper into server virtualization

LEARNING OBJECTIVES (SLIDE 3)

1. Explain **virtualization** and the role of the **hypervisor**: how it abstracts physical hardware to run multiple operating systems on one machine.
2. Differentiate **Type 1 and Type 2** hypervisors: architecture, examples, performance trade-offs and typical use cases.
3. Compare **virtual machines and containers**: the architectural difference, and choose the right technology for a given workload.
4. Describe **cloud data centre architecture**: regions, availability zones, resource pooling, multi-tenancy and orchestration.

1. What is cloud infrastructure? (slide 4)

- **Definition:** the hardware and software components — **servers, storage, networking and virtualization** — required to support cloud computing.
- **Delivered over the Internet:** resources are provisioned and accessed remotely, on demand, without the customer owning the physical equipment.
- **The foundation for cloud services:** it underpins IaaS, PaaS and SaaS from AWS, Azure and Google Cloud.

2. Virtualization concepts (slides 5–8)

VIRTUALIZATION (DEFINITION, SLIDE 6)

Creating a **virtual (software-based) version of a resource** — a server, storage device, network or operating system. It lets **one physical machine behave as if it were several independent machines**, and it is the **core enabling technology of the cloud**: nearly every cloud service sits on a virtualization layer that shares physical hardware across many customers.

Life before virtualization (slide 7)

- **One application per physical server** (1990s–early 2000s): each server ran one OS and one application, to keep apps from interfering with each other.
- **Very low utilization:** servers ran at only **10–15% average CPU**, wasting the rest.
- **Expensive to scale:** more capacity meant buying, racking, powering and cooling another whole machine.

Worked example: consolidation

Before	After virtualization
10 physical servers, each one app, each at ~12% CPU. Total useful work = 10 × 12% = **1.2 servers' worth**.	Run the 10 apps as 10 VMs on **2 hosts**: each host carries 5 × 12% = 60% (plus a little hypervisor overhead). Leaves headroom for spikes, and if one host fails its VMs can restart on the other (with reduced headroom).
Savings: 8 fewer machines to buy, power, cool, rack and maintain. This is **server consolidation**.	

What is a hypervisor? (slide 8)

- Also called a **Virtual Machine Monitor (VMM)**: software that **creates, runs and manages virtual machines**.
- It **sits between hardware and VMs**: it allocates physical CPU, memory, storage and network resources to each VM and **keeps them isolated** from each other.
- Two categories: **Type 1 (bare-metal)** and **Type 2 (hosted)**, depending on what they run on.

3. Type 1 vs Type 2 hypervisors (slides 9–11)

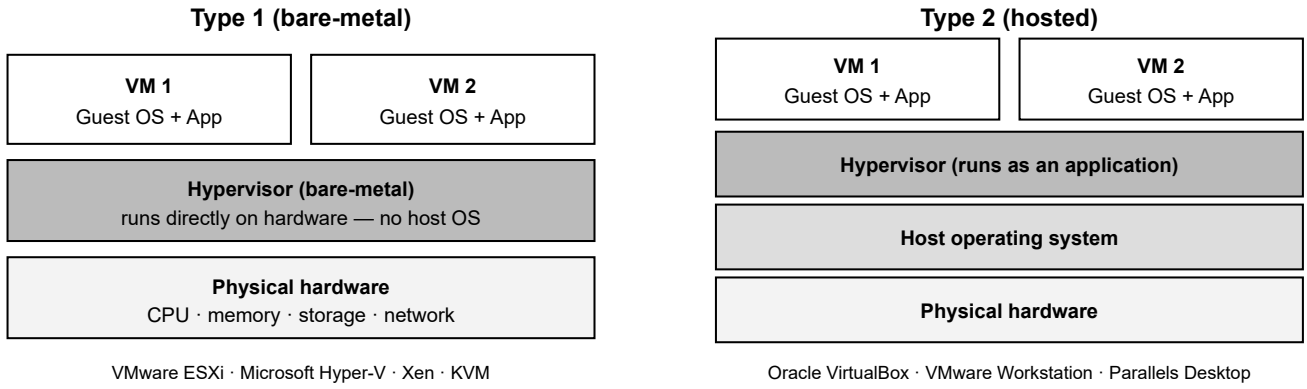


Figure 1 — Type 1 runs on the hardware; Type 2 runs as an app on a host OS, so every hardware call passes through one more layer.

Aspect (slide 11)	Type 1 (bare-metal)	Type 2 (hosted)
Runs on	Physical hardware directly	A host operating system (Windows, macOS, Linux)
Performance	Near-native, low overhead: talks to hardware directly	Lower: extra OS layer; every hardware call passes through the host OS as well as the hypervisor
Installation	More involved — replaces the OS	Simple — installed like any app
Typical use	Enterprise data centres and cloud platforms	Developer laptops: testing, sandboxing, personal use
Security	Smaller attack surface (no general-purpose host OS to secure)	Inherits host OS vulnerabilities
Examples	VMware ESXi, Hyper-V, Xen, KVM	VirtualBox, VMware Workstation, Parallels

Lab 01 uses VMware **ESXi**, a Type 1 hypervisor. If you run Linux in VirtualBox on your laptop, that is Type 2.

How hypervisors virtualize resources (slide 12)

Resource	How it is virtualized	Beginner picture
CPU	Physical CPU cores are **time-sliced** and scheduled across virtual CPUs (vCPUs), one or more per VM	One chef cooking for several tables in turns, so fast that each table feels it has its own chef
Memory	Physical RAM is **partitioned and mapped** so each VM has its own isolated, addressable memory space	Each VM thinks its RAM starts at address 0; the hypervisor translates to the real location
I/O	**Virtual NICs and virtual disks** stand in for physical network cards and storage, translating requests to the real hardware	A VM's "disk" is really a large file on the host's storage

4. Types of virtualization (slides 13–16)

Type	What it does	Key points
Server	Multiple virtual servers run on one physical machine, each with its own OS	Each VM runs its own guest OS and apps, unaware of the others. **Consolidation**: many underused servers onto fewer, busier machines. **Isolation**: a crash in one VM does not directly affect others on the same host.
Storage	Physical storage devices are **pooled into one logical storage resource**	Disks and arrays from many devices form one pool. **Simplified management**: capacity managed as one resource. **Advanced features**: snapshots, replication, **thin provisioning** (give a VM a 100 GB disk but only use real space as data is written), easier backup and DR.
Network	Network resources abstracted into **virtual switches, VLANs and software-defined networks**	Virtual switches/routers/VLANs created independently of physical hardware. **SDN**: network control centralized in software, separate from forwarding devices (Lesson 3). **Flexible**: create, reconfigure or tear down networks in software, without rewiring cables.
Desktop	Desktop OSes run centrally and are **delivered remotely** to end-user devices	**VDI** = Virtual Desktop Infrastructure (Citrix, Dialog DaaS in Lesson 3).

A **VLAN** (virtual LAN) splits one physical switch into several separate logical networks, so traffic from different groups stays apart.

5. Benefits and limitations of virtualization (slides 17–18)

Benefit	Why
Better hardware utilization	Many workloads share the same physical resources instead of sitting idle
Cost savings	Fewer physical servers: less spending on hardware, power and cooling
Faster provisioning	New VMs in minutes instead of ordering and racking hardware
Easier disaster recovery	VM snapshots and images can be backed up, copied and restored quickly

Limitation	Why
Performance overhead	The hypervisor layer adds some CPU, memory and I/O overhead vs bare hardware
Licensing complexity	Software licences were not always designed for virtualized, movable workloads
Single point of failure	If the physical host or hypervisor fails, **every VM on it** is affected at once
Security risks	A compromised hypervisor can expose every VM it hosts — **"hyperjacking"**

6. Virtual machines vs containers (slides 19–28)

6.1 What is a virtual machine? (slides 20–21)

- **A complete virtual computer:** a full guest OS, virtual CPU, memory, storage and network interfaces.
- **Managed by a hypervisor**, which allocates physical resources and keeps it isolated from other VMs.
- **Strong isolation, heavier footprint:** independent and secure, but carries the size and boot time of a full OS (minutes to boot).

VM stack (slide 21): physical hardware → hypervisor → guest OS (full OS: kernel, drivers, libraries) → application.

6.2 What is a container? (slides 22–23)

- **A lightweight, portable package:** the application plus just the code, runtime and libraries it needs.
- **Shares the host OS kernel:** no OS of its own; it uses the kernel of the host it runs on.
- **Managed by a container engine** such as Docker, on top of the host OS. Seconds to start.

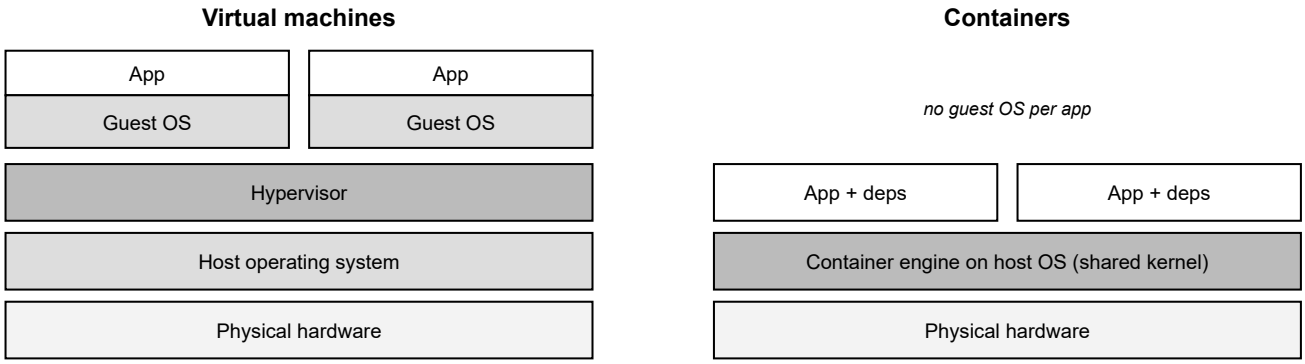


Figure 2 — Architecture compared (slide 24). Slide 24 draws the VM side with a host OS below the hypervisor (Type 2 style); with a Type 1 hypervisor that layer is absent.

Metric (slide 25)	Virtual machines	Containers
Isolation	**Strong** — separate kernel	**Weaker** — shared kernel
Size	Gigabytes	Megabytes
Boot time	Minutes	Seconds
Density per host	Lower	Higher
Typical use	Strict isolation, mixed OS needs	Microservices, fast scaling, CI/CD

Worked example (density): a host with 64 GB RAM. Each VM needs ~2 GB for its guest OS + 1 GB for the app = 3 GB ⇒ about 21 VMs. Each container needs only the app's ~1 GB (the kernel is shared) ⇒ about 60 containers. Same hardware, roughly 3× the apps.

6.3 Docker and container engines (slide 26)

Docker is the most widely used container platform: tools to build, ship and run containers. A Docker **image** is a read-only template; a **container** is a running instance created from that image. A **Dockerfile** scripts how an image is built; **registries** such as Docker Hub store and distribute images (full detail in Lesson 3).

6.4 Choosing VMs vs containers (slide 27)

Use virtual machines when…

- Strong, hardware-level isolation between workloads is required.
- Different guest operating systems must run on the same host (e.g. Windows and Linux).
- Legacy applications depend on a specific full OS environment.
- Multi-tenant security boundaries are the top priority.

Use containers when…

- The workload is built as microservices.
- Fast startup and high deployment density matter.
- Applications are packaged and shipped through CI/CD pipelines.
- Elastic, rapid horizontal scaling is required.

6.5 Hybrid: containers inside VMs (slide 28)

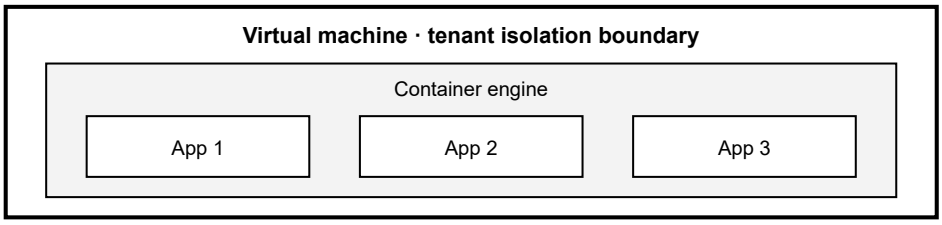


Figure 3 — Cloud providers commonly run containers inside a VM: the VM gives strong, hardware-enforced isolation between tenants; the containers inside give fast, efficient deployment.

7. Cloud data centre architecture (slides 29–38)

7.1 Data centre and its physical components (slides 30–31)

A **data centre** is a physical facility that houses the compute, storage and networking infrastructure that powers cloud services. Major providers (AWS, Azure, Google Cloud) run global networks of their own; every virtual resource a customer uses ultimately runs on hardware in one of them.

Component	Role
Servers and racks	Compute hardware organised into racks and clusters
Storage arrays	Disk and flash storage holding customer and system data
Networking gear	Switches, routers and cabling connecting servers to each other and the Internet
Power and cooling	**UPS** (uninterruptible power supply: batteries for short outages), generators, cooling to keep hardware running reliably

7.2 Regions and availability zones (slide 32)

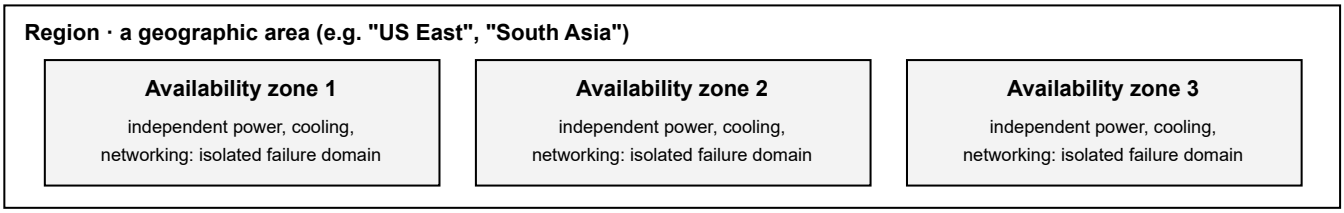


Figure 4 — Workloads are spread across zones so a single facility failure does not take the whole application offline.

A **failure domain** is a part of the system that fails as one unit (one power feed, one building). AZs are separate failure domains.

7.3 Resource pooling and multi-tenancy (slides 33–34)

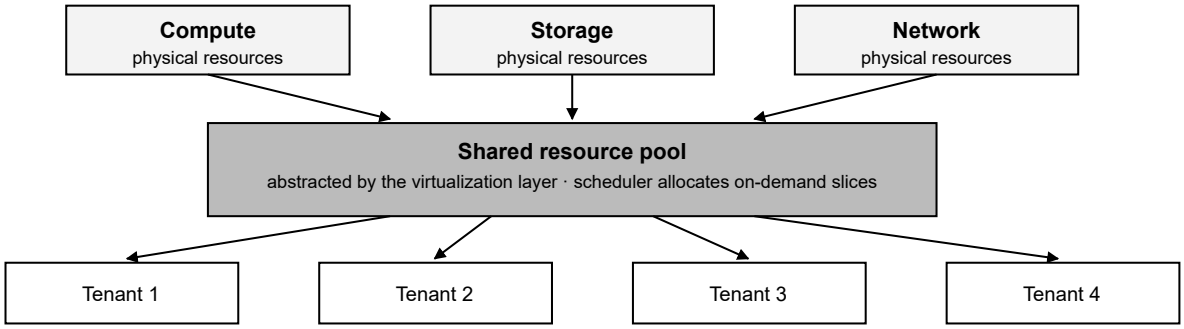


Figure 5 — Resource pooling (slide 33).

Multi-tenancy (slide 34): (1) **shared infrastructure, separate customers**: many tenants run on the same physical infrastructure at the same time; (2) **isolation is enforced in software**: the virtualization layer keeps each tenant's compute, storage and data separate; (3) **economies of scale**: sharing across many tenants is what makes low, pay-as-you-go prices possible.

7.4 Cloud architecture layers (slide 35)

Layer (top to bottom)	Role
Service layer	IaaS, PaaS, SaaS — what customers consume
Orchestration & management	Schedulers decide placement, scaling and recovery
Virtualization layer	Hypervisors and container engines pool and allocate resources
Physical infrastructure	Servers, storage, networking, power, cooling

7.5 Orchestration: Kubernetes (slide 36)

An open-source platform that automates the **deployment, scaling and management of containerized applications**. **Scheduling:** decides which physical or virtual machine each container runs on, based on available resources. **Self-healing and scaling:** restarts failed containers, scales replicas up or down, and redistributes workloads after a failure (Lesson 6).

7.6 Elasticity and on-demand scaling (slide 37)

- **Dynamic reassignment:** pooled resources are reassigned to workloads automatically as demand rises and falls.
- **Auto-scaling:** new VM or container instances are added under load and removed when demand drops.
- **Pay-as-you-go:** customers pay only for resources actually consumed, not fixed owned capacity.

7.7 Real-world providers (slide 38)

Provider	Underlying virtualization	Global footprint
Amazon Web Services	Custom KVM-based **Nitro** hypervisor	Regions and availability zones worldwide
Microsoft Azure	Microsoft **Hyper-V**	Regions with **paired-region** redundancy
Google Cloud	**KVM**-based hypervisor	Regions and zones across continents

8. Summary (slide 39)

1. **Virtualization concepts:** hypervisors (Type 1 bare-metal vs Type 2 hosted) abstract hardware so multiple OSes share one machine.
2. **VMs vs containers:** VMs virtualize the **hardware** with a full guest OS; containers virtualize the **OS** and share the host kernel.
3. **Cloud data centre architecture:** providers pool physical resources across regions and AZs and allocate them on demand to many tenants.

9. Exam-style questions with model answers

Q1. Define virtualization and a hypervisor. Why is virtualization called the core enabling technology of the cloud? (5 marks)
Virtualization creates a software-based version of a resource (server, storage, network, OS) so one physical machine behaves like several independent machines. A hypervisor (virtual machine monitor) is software that creates, runs and manages VMs, sitting between hardware and VMs, allocating CPU, memory, storage and network and isolating the VMs. It is the core enabler because nearly every cloud service runs on a virtualization layer that shares physical hardware across many customers, enabling resource pooling, multi-tenancy, rapid provisioning and elasticity.

Q2. Compare Type 1 and Type 2 hypervisors with examples. (8)
Type 1 (bare-metal) runs directly on the hardware with no host OS: near-native performance, smaller attack surface, more involved installation, used in data centres and clouds (ESXi, Hyper-V, Xen, KVM). Type 2 (hosted) runs as an application on a host OS: easy to install, but lower performance because calls pass through the host OS, and it inherits host OS vulnerabilities; used on developer laptops for testing and sandboxing (VirtualBox, VMware Workstation, Parallels).

Q3. Explain how a hypervisor virtualizes CPU, memory and I/O. (6)
CPU: physical cores are time-sliced and scheduled across virtual CPUs given to each VM. Memory: physical RAM is partitioned and mapped so each VM has its own isolated address space. I/O: virtual NICs and virtual disks stand in for physical devices and translate VM requests to the real hardware.

Q4. Describe four types of virtualization. (8)
Server: many virtual servers with their own OSes on one physical server (consolidation, isolation). Storage: physical storage pooled into one logical resource (simpler management, snapshots, replication, thin provisioning). Network: virtual switches, VLANs and SDN abstract the network so it can be changed in software. Desktop: desktop OSes run centrally and are delivered remotely (VDI).

Q5. Give the benefits and limitations of virtualization. (6)
Benefits: better utilisation, cost savings (hardware, power, cooling), faster provisioning, easier disaster recovery through snapshots/images. Limitations: hypervisor performance overhead, licensing complexity, the host/hypervisor is a single point of failure for all its VMs, and a compromised hypervisor exposes all VMs (hyperjacking).

Q6. Compare VMs and containers on isolation, size, boot time, density and use. When would you choose each? (8)
VMs: strong isolation (separate kernel), GBs, minutes to boot, lower density, used for strict isolation and mixed OS needs. Containers: weaker isolation (shared kernel), MBs, seconds to start, higher density, used for microservices, fast scaling and CI/CD. Choose VMs for hardware-level isolation, different guest OSes, legacy apps needing a full OS, and multi-tenant security. Choose containers for microservices, fast startup and density, CI/CD pipelines and rapid horizontal scaling.

Q7. Why do cloud providers run containers inside VMs? (3)
The VM provides strong, hardware-enforced isolation between tenants, while containers inside it provide fast, efficient deployment — combining the security of VMs with the agility of containers.

Q8. Explain regions, availability zones, resource pooling and multi-tenancy. (8)
A region is a geographic area containing several availability zones. Each AZ has independent power, cooling and networking and is an isolated failure domain, so workloads spread across AZs survive a facility failure. Resource pooling combines compute, storage and network into a shared pool through virtualization, from which a scheduler allocates on-demand slices. Multi-tenancy means many customers share the same infrastructure with isolation enforced in software, giving economies of scale and low pay-as-you-go prices.

Q9. Describe the four layers of cloud architecture. (4)
Physical infrastructure (servers, storage, networking, power, cooling) → virtualization layer (hypervisors and container engines pool and allocate) → orchestration and management (schedulers decide placement, scaling, recovery) → service layer (IaaS, PaaS, SaaS consumed by customers).

10. Common mistakes

Mistake	Correct idea
Type 1 runs on top of Windows	Type 1 runs on bare hardware; Type 2 runs on a host OS.
VirtualBox is a Type 1 hypervisor	VirtualBox, Workstation and Parallels are Type 2.
Containers virtualize hardware	Containers virtualize the OS; VMs virtualize the hardware.
Containers are more isolated than VMs	VMs have stronger isolation (separate kernel).
Virtualization removes single points of failure by itself	The host/hypervisor is a SPOF for all its VMs; you still need multiple hosts/AZs.
Region = availability zone	A region contains several AZs.
Multi-tenancy means tenants can see each other's data	Isolation is enforced in software by the virtualization layer.
Hyperjacking = attacking one VM	It is compromising the hypervisor, which exposes all VMs.

Cheat sheet

Virtualization	
Definition	Software version of a resource; 1 machine acts as many; core cloud enabler
Before	1 app/server, 10–15% CPU, costly scaling
Hypervisor	= VMM; creates, runs, manages VMs; allocates + isolates
CPU / Mem / I/O	Time-sliced vCPUs / partitioned mapped RAM / vNICs + virtual disks
Types	Server, storage (pool, thin provisioning), network (vSwitch, VLAN, SDN), desktop (VDI)
Benefits	Utilisation, cost, fast provisioning, easy DR
Limits	Overhead, licensing, host SPOF, hyperjacking

	Type 1	**Type 2**
On	Hardware	Host OS
Perf.	Near-native	Lower
Install	Replaces OS	Like an app
Use	DCs, cloud	Laptops, testing
Security	Small surface	Host OS risks
e.g.	ESXi, Hyper-V, Xen, KVM	VirtualBox, Workstation, Parallels

	VM	**Container**
Virtualizes	Hardware	OS
Isolation	Strong	Weaker
Size	GB	MB
Boot	Minutes	Seconds
Density	Lower	Higher
Use	Isolation, mixed OS, legacy, multi-tenant	Microservices, CI/CD, fast scaling
Hybrid: containers inside VMs (tenant isolation + agility)		

Data centre	
Parts	Servers/racks, storage arrays, networking, power (UPS, generators) + cooling
Region / AZ	Geographic area / isolated failure domain with own power, cooling, network
Pooling	Compute + storage + network → shared pool → scheduler slices → tenants
Multi-tenancy	Shared infra, software isolation, economies of scale
Layers	Service → orchestration → virtualization → physical
Kubernetes	Schedules containers, self-heals, scales
Elasticity	Dynamic reassignment, auto-scaling, pay-as-you-go
Providers	AWS Nitro (KVM) · Azure Hyper-V (paired regions) · GCP KVM